



MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Petcov Nicolai FAF-233

Report

Laboratory Work No. 5

Information Security and Cryptography

Public Key Cryptography: RSA, ElGamal, and Diffie-Hellman

Checked by:
Zaica M., *university assistant*
DISA, FCIM, UTM

Chişinău – 2025

1 Objective

Study and implement public key cryptography algorithms including RSA, ElGamal, and Diffie-Hellman key exchange. Understand asymmetric encryption principles, the mathematical foundations of modern cryptography, and develop practical skills in implementing cryptographic protocols with large prime numbers and modular arithmetic.

2 Tasks

1. Implement RSA encryption/decryption with 2048-bit modulus, encrypt message "Nicolai Petcov" via ASCII hexadecimal to decimal conversion
2. Implement ElGamal encryption/decryption using provided 2048-bit prime p and generator $g = 2$, encrypt same message
3. Implement Diffie-Hellman key exchange, derive AES-256 key from shared secret, demonstrate AES-256 encryption/decryption
4. Provide step-by-step explanations of all algorithms

3 Theoretical Background

3.1 Public Key Cryptography

Public key cryptography uses two mathematically related keys: a public key for encryption and a private key for decryption. Key advantages include eliminating secure key exchange channels and enabling digital signatures, though asymmetric operations are significantly slower than symmetric encryption.

3.2 Mathematical Foundations

Modular Arithmetic: For integers a, b, n : $a \equiv b \pmod{n}$. Operations maintain congruence under addition, multiplication, and exponentiation.

Euler's Totient: $\phi(n)$ counts integers coprime to n . For primes: $\phi(p) = p - 1$. For RSA modulus: $\phi(pq) = (p - 1)(q - 1)$.

Euler's Theorem: If $\gcd(a, n) = 1$, then $a^{\phi(n)} \equiv 1 \pmod{n}$.

Modular Inverse: $a \times a^{-1} \equiv 1 \pmod{n}$, computed via Extended Euclidean Algorithm.

Discrete Logarithm: Finding x from $y = g^x \pmod{p}$ is computationally hard, forming the basis of ElGamal and Diffie-Hellman security.

3.3 RSA Algorithm

RSA (Rivest-Shamir-Adleman, 1977) security is based on the difficulty of factoring large composite numbers.

Key Generation:

1. Generate large primes p and q (≈ 1024 bits each for 2048-bit modulus)

2. Compute $n = p \times q$ and $\phi(n) = (p - 1)(q - 1)$
3. Choose $e = 65537$ where $\gcd(e, \phi(n)) = 1$
4. Compute $d \equiv e^{-1} \pmod{\phi(n)}$
5. Public key: (e, n) ; Private key: (d, n)

Encryption: $C = M^e \pmod{n}$

Decryption: $M = C^d \pmod{n}$

Correctness: Since $ed \equiv 1 \pmod{\phi(n)}$, we have $C^d = M^{ed} = M^{k\phi(n)+1} = M \pmod{n}$ by Euler's theorem.

3.4 ElGamal Algorithm

ElGamal (1985) is based on the discrete logarithm problem and provides semantic security through randomized encryption.

Key Generation: Given public p, g :

1. Choose random private key x where $1 < x < p - 1$
2. Compute public key $y = g^x \pmod{p}$

Encryption: For message M :

1. Choose random k where $1 < k < p - 1$ (must be unique per encryption!)
2. Compute $c_1 = g^k \pmod{p}$ and $c_2 = M \times y^k \pmod{p}$
3. Ciphertext: (c_1, c_2)

Decryption:

1. Compute $s = c_1^x \pmod{p}$ (note: $s = y^k$)
2. Compute $M = c_2 \times s^{-1} \pmod{p}$

3.5 Diffie-Hellman Key Exchange

Diffie-Hellman (1976) allows two parties to establish a shared secret over an insecure channel.

Protocol: Given public p, g :

1. Alice: choose secret a , compute $A = g^a \pmod{p}$, send A
2. Bob: choose secret b , compute $B = g^b \pmod{p}$, send B
3. Alice computes: $K = B^a \pmod{p} = g^{ab} \pmod{p}$
4. Bob computes: $K = A^b \pmod{p} = g^{ab} \pmod{p}$

Shared secret K is then hashed (e.g., SHA-256) to derive an AES-256 key.

Security: Eavesdroppers see p, g, A, B but cannot compute g^{ab} without solving the discrete logarithm problem. Note: basic DH lacks authentication and is vulnerable to man-in-the-middle attacks.

3.6 AES-256 Encryption

AES (Advanced Encryption Standard) is a symmetric block cipher with 128-bit blocks. AES-256 uses 256-bit keys (14 rounds).

CBC Mode: Cipher Block Chaining XORs each plaintext block with the previous ciphertext block before encryption. First block uses random Initialization Vector (IV).

Padding: PKCS#7 padding ensures messages are multiples of block size.

4 Implementation

4.1 Text to Decimal Conversion

```
1 def text_to_decimal(text):
2     """
3     Convert text to decimal via ASCII hexadecimal
4
5     Example:
6     "ABC" -> "414243" (hex) -> 4276803 (decimal)
7     """
8     hex_string = text.encode('ascii').hex()
9     decimal_value = int(hex_string, 16)
10
11     print(f"Text: {text}")
12     print(f"Hex: {hex_string}")
13     print(f"Decimal: {decimal_value}")
14
15     return decimal_value
16
17 def decimal_to_text(decimal_value):
18     """
19     Convert decimal back to text
20     """
21     hex_string = hex(decimal_value)[2:]
22
23     if len(hex_string) % 2 != 0:
24         hex_string = '0' + hex_string
25
26     text = bytes.fromhex(hex_string).decode('ascii')
27     return text
```

Listing 1: Text Conversion Functions

4.2 RSA Implementation

4.2.1 Prime Number Generation

```
1 import random
2 from sympy import isprime
3
4 def generate_large_prime(bits):
5     """
6     Generate a prime number with specified bit length
7     """
8     while True:
```

```

9      # Generate random number of required size
10     candidate = random.getrandbits(bits)
11
12     # Ensure highest bit is 1 (correct bit length)
13     candidate |= (1 << (bits - 1))
14
15     # Ensure number is odd
16     candidate |= 1
17
18     # Test for primality
19     if isprime(candidate):
20         return candidate

```

Listing 2: Large Prime Generation

4.2.2 Key Generation

```

1  from sympy import gcd, mod_inverse
2
3  def generate_rsa_keys(bits=2048):
4      """
5      Generate RSA public and private keys
6
7      Returns:
8          public_key: (e, n)
9          private_key: (d, n)
10     """
11     # Step 1: Generate p and q
12     half_bits = bits // 2
13     p = generate_large_prime(half_bits)
14     q = generate_large_prime(half_bits)
15
16     # Step 2: Compute n = p * q
17     n = p * q
18
19     # Step 3: Compute phi(n) = (p-1)(q-1)
20     phi_n = (p - 1) * (q - 1)
21
22     # Step 4: Choose e = 65537
23     e = 65537
24
25     # Verify gcd(e, phi(n)) = 1
26     if gcd(e, phi_n) != 1:
27         return generate_rsa_keys(bits)
28
29     # Step 5: Compute d = e^(-1) mod phi(n)
30     d = mod_inverse(e, phi_n)
31
32     # Verify: (d * e) mod phi(n) = 1
33     assert (d * e) % phi_n == 1
34
35     return (e, n), (d, n)

```

Listing 3: RSA Key Generation

4.2.3 Encryption and Decryption

```

1 def rsa_encrypt(message, public_key):
2     """
3     Encrypt message using RSA public key
4
5      $C = M^e \bmod n$ 
6     """
7     e, n = public_key
8
9     # Check message size
10    if message >= n:
11        raise ValueError("Message too large for modulus")
12
13    # Fast modular exponentiation
14    ciphertext = pow(message, e, n)
15
16    return ciphertext
17
18 def rsa_decrypt(ciphertext, private_key):
19     """
20     Decrypt ciphertext using RSA private key
21
22      $M = C^d \bmod n$ 
23     """
24     d, n = private_key
25
26     # Fast modular exponentiation
27     plaintext = pow(ciphertext, d, n)
28
29     return plaintext

```

Listing 4: RSA Encryption/Decryption

4.3 ElGamal Implementation

4.3.1 Key Generation

```

1 # Given parameters (2048-bit prime)
2 P = 3231700607131100730015351347782516336248805713348907517458... #
3   Full value
4 G = 2
5
6 def generate_elgamal_keys(p, g):
7     """
8     Generate ElGamal public and private keys
9
10    Returns:
11        public_key: (p, g, y)
12        private_key: x
13    """
14    # Choose random private key x
15    x = random.randint(2, p - 2)
16
17    # Compute public key  $y = g^x \bmod p$ 
18    y = pow(g, x, p)
19
20    return (p, g, y), x

```

Listing 5: ElGamal Key Generation

4.3.2 Encryption and Decryption

```
1 def elgamal_encrypt(message, public_key):
2     """
3     Encrypt message using ElGamal
4
5     Returns: (c1, c2)
6     """
7     p, g, y = public_key
8
9     # Choose random k
10    k = random.randint(2, p - 2)
11
12    # Compute c1 = g^k mod p
13    c1 = pow(g, k, p)
14
15    # Compute c2 = M * y^k mod p
16    y_k = pow(y, k, p)
17    c2 = (message * y_k) % p
18
19    return c1, c2
20
21 def elgamal_decrypt(ciphertext, private_key, p):
22     """
23     Decrypt ElGamal ciphertext
24
25     Returns: plaintext M
26     """
27    c1, c2 = ciphertext
28    x = private_key
29
30    # Compute s = c1^x mod p
31    s = pow(c1, x, p)
32
33    # Compute s^(-1) mod p
34    s_inv = mod_inverse(s, p)
35
36    # Compute M = c2 * s^(-1) mod p
37    plaintext = (c2 * s_inv) % p
38
39    return plaintext
```

Listing 6: ElGamal Encryption/Decryption

4.4 Diffie-Hellman Implementation

4.4.1 Key Exchange

```
1 def diffie_hellman_exchange(p, g):
2     """
3     Perform Diffie-Hellman key exchange
4
5     Returns:
6         shared_key: K = g^(ab) mod p
7         a, b: secret numbers
8         A, B: public values
9     """
```

```
10 # Alice chooses secret a
11 a = random.randint(2, p - 2)
12
13 # Bob chooses secret b
14 b = random.randint(2, p - 2)
15
16 # Alice computes A = g^a mod p
17 A = pow(g, a, p)
18
19 # Bob computes B = g^b mod p
20 B = pow(g, b, p)
21
22 # Alice computes shared key: K = B^a mod p
23 K_alice = pow(B, a, p)
24
25 # Bob computes shared key: K = A^b mod p
26 K_bob = pow(A, b, p)
27
28 # Verify keys match
29 assert K_alice == K_bob
30
31 return K_alice, a, b, A, B
```

Listing 7: Diffie-Hellman Key Exchange

4.4.2 AES Key Derivation

```
1 import hashlib
2
3 def derive_aes_key(shared_secret):
4     """
5     Derive AES-256 key from Diffie-Hellman shared secret
6
7     Returns: 32-byte AES key
8     """
9     # Convert shared secret to bytes
10    secret_bytes = shared_secret.to_bytes(
11        (shared_secret.bit_length() + 7) // 8,
12        byteorder='big'
13    )
14
15    # Apply SHA-256 to get exactly 256 bits
16    aes_key = hashlib.sha256(secret_bytes).digest()
17
18    return aes_key
```

Listing 8: Derive AES-256 Key from DH Shared Secret

4.4.3 AES-256 Encryption

```
1 from Crypto.Cipher import AES
2 from Crypto.Util.Padding import pad, unpad
3 from Crypto.Random import get_random_bytes
4
5 def aes_encrypt(plaintext, key):
6     """
```

```

7   Encrypt plaintext using AES-256 CBC mode
8
9   Returns: (ciphertext, iv)
10  """
11  # Generate random IV (16 bytes for AES)
12  iv = get_random_bytes(16)
13
14  # Create cipher object
15  cipher = AES.new(key, AES.MODE_CBC, iv)
16
17  # Convert text to bytes and add padding
18  plaintext_bytes = plaintext.encode('utf-8')
19  padded = pad(plaintext_bytes, AES.block_size)
20
21  # Encrypt
22  ciphertext = cipher.encrypt(padded)
23
24  return ciphertext, iv
25
26 def aes_decrypt(ciphertext, key, iv):
27     """
28     Decrypt ciphertext using AES-256 CBC mode
29
30     Returns: plaintext
31     """
32     # Create cipher object with same IV
33     cipher = AES.new(key, AES.MODE_CBC, iv)
34
35     # Decrypt
36     padded = cipher.decrypt(ciphertext)
37
38     # Remove padding
39     plaintext_bytes = unpad(padded, AES.block_size)
40     plaintext = plaintext_bytes.decode('utf-8')
41
42     return plaintext

```

Listing 9: AES-256 CBC Mode Encryption

5 Results

5.1 RSA Results

Input: Message "Nicolai Petcov" → Hex: 4E69636F6C616920506574636F76 → Decimal: 409103023002501386547876469110134

Keys Generated: 2048-bit modulus n , $e = 65537$, private exponent d

Process: $C = M^{65537} \bmod n \rightarrow M = C^d \bmod n$

Result: Successfully recovered "Nicolai Petcov"

5.2 ElGamal Results

Parameters: 2048-bit prime p , generator $g = 2$

Input: Same message (decimal: 409103023002501386547876469110134)

Keys: Random private x , public $y = g^x \bmod p$

Process: Random k chosen, (c_1, c_2) computed, decrypted via $s = c_1^x$, $M = c_2 \times s^{-1} \bmod p$

Result: Successfully recovered "Nicolai Petcov"

5.3 Diffie-Hellman + AES Results

Key Exchange: Alice secret a , public $A = g^a \bmod p$; Bob secret b , public $B = g^b \bmod p$

Shared Secret: $K = g^{ab} \bmod p$ (verified: $B^a = A^b$)

AES Key: Derived via $\text{SHA-256}(K) = 32$ bytes

AES Test: Encrypted "This message is encrypted with AES-256" with random IV, successfully decrypted

6 Security Analysis

6.1 Algorithm Comparison

Feature	RSA	ElGamal	DH
Use case	Encryption/Signatures	Encryption	Key exchange
Security basis	Factorization	Discrete log	Discrete log
Ciphertext size	$1 \times$ modulus	$2 \times$ message	N/A
Randomized	No	Yes	N/A

6.2 Key Security Considerations

RSA: 2048-bit keys secure until ~ 2030 . Vulnerable to quantum computers (Shor's algorithm). Requires proper padding (OAEP) and constant-time implementations to prevent timing attacks.

ElGamal: Provides semantic security through randomization. Critical: never reuse ephemeral key k . Ciphertext expansion ($2 \times$ size) is a disadvantage.

Diffie-Hellman: Enables forward secrecy but lacks authentication. Vulnerable to man-in-the-middle attacks without additional authentication (signatures/certificates).

AES-256: Post-quantum resistant (Grover's algorithm reduces effective security to 128 bits). Current standard for symmetric encryption.

6.3 Modern Applications

Hybrid Cryptography: Modern systems use DH/RSA for key exchange, AES for bulk encryption (e.g., TLS 1.3, PGP).

Post-Quantum: NIST standardizing CRYSTALS-Kyber (key encapsulation) and CRYSTALS-Dilithium (signatures) to replace RSA/DH.

7 Conclusions

Successfully implemented three foundational public-key algorithms:

1. **RSA:** 2048-bit encryption/decryption with proper key generation and modular exponentiation
2. **ElGamal:** Probabilistic encryption demonstrating semantic security
3. **Diffie-Hellman:** Secure key exchange with AES-256 derivation

Key Learning Outcomes:

- Mathematical foundations: modular arithmetic, Euler's theorem, discrete logarithm
- Practical cryptographic implementation with large integers (2048+ bits)
- Security trade-offs: asymmetric vs symmetric, deterministic vs probabilistic
- Hybrid encryption combining public-key and symmetric cryptography

Critical Insights:

- Key size requirements: 2048-bit minimum for RSA/DH in 2025
- Cryptographic randomness is essential for security
- Quantum threat necessitates post-quantum cryptography
- Modern systems use hybrid approaches (DH + AES) for optimal security and performance

These algorithms remain fundamental to understanding modern cryptographic protocols (TLS, VPNs, secure messaging) and the evolution toward post-quantum systems.

Source Code

<https://github.com/Nickseen/Cryptology-Security/tree/Lab5>